Faculty of Electronics, Telecommunications and Information Technology

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA ROMANIA

Color Sorting Machine

~ Using TCS3200 color sensor and a servo motor/step motor controlled by an Arduino board to sort colored objects~

Student:
Negrea Sara-Diana
Group 2031/3, English series

Coordinator:
Ph.D.Eng Mircea Giurgiu
Lab. Assistant Maeva Cărbunescu

1.Introduction

1.1 Requirements

For my Microprocessor final Mini Project, I had to implement a Color Sorting Machine using a TCS3200 sensor, a servo motor/step motor , which in my case tuned into 2 servo motors of type SG90, all controlled by an Arduino board (Arduino UNO R3).

Why choose servo motor and not step motor for this project?

After taking my time to see which one would fit for my design and implementation of the project, I concluded that the best option would be the SG90 SERVO MOTOR, because:

- Has built-in position feedback
- Doesn't need a driver like in the case of a STEP motor, which is much more complex for this task
- Lightweight, easy to use for beginners and not so complex setup

1.2 DESINGING

For the desing I got inspired from multiple online videos and I tried to make it as realistic as possible, so that it actually looks more like an industrial sorting machine, and for the objects sorted, I chose M&M candy, because it is lightweight and comes in different colors. For actually implementing it I used AutoCAD to draw all the parts of the box and to scale it as precise as possible, in order to assemble it then.

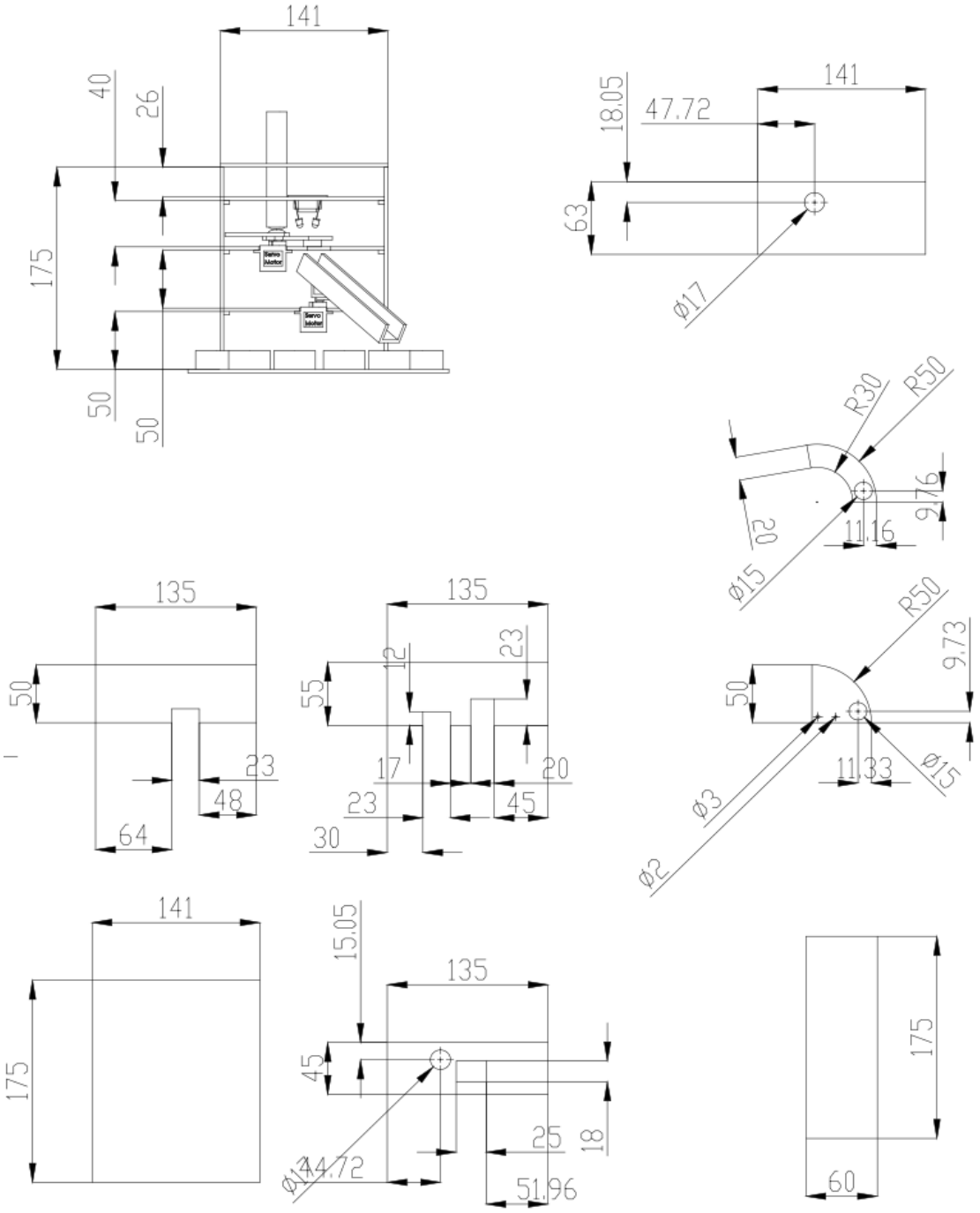


Figure 1- AutoCAD drawing with dimensions

As for the material used, I chose a 4mm tick cardboard, redraw on it all the pieces of the machine and carefully cut them and assembled using a hot glue gun. The pipe that stores the candy before sorting is made of a flexible plastic that was rolled and glued to the box.

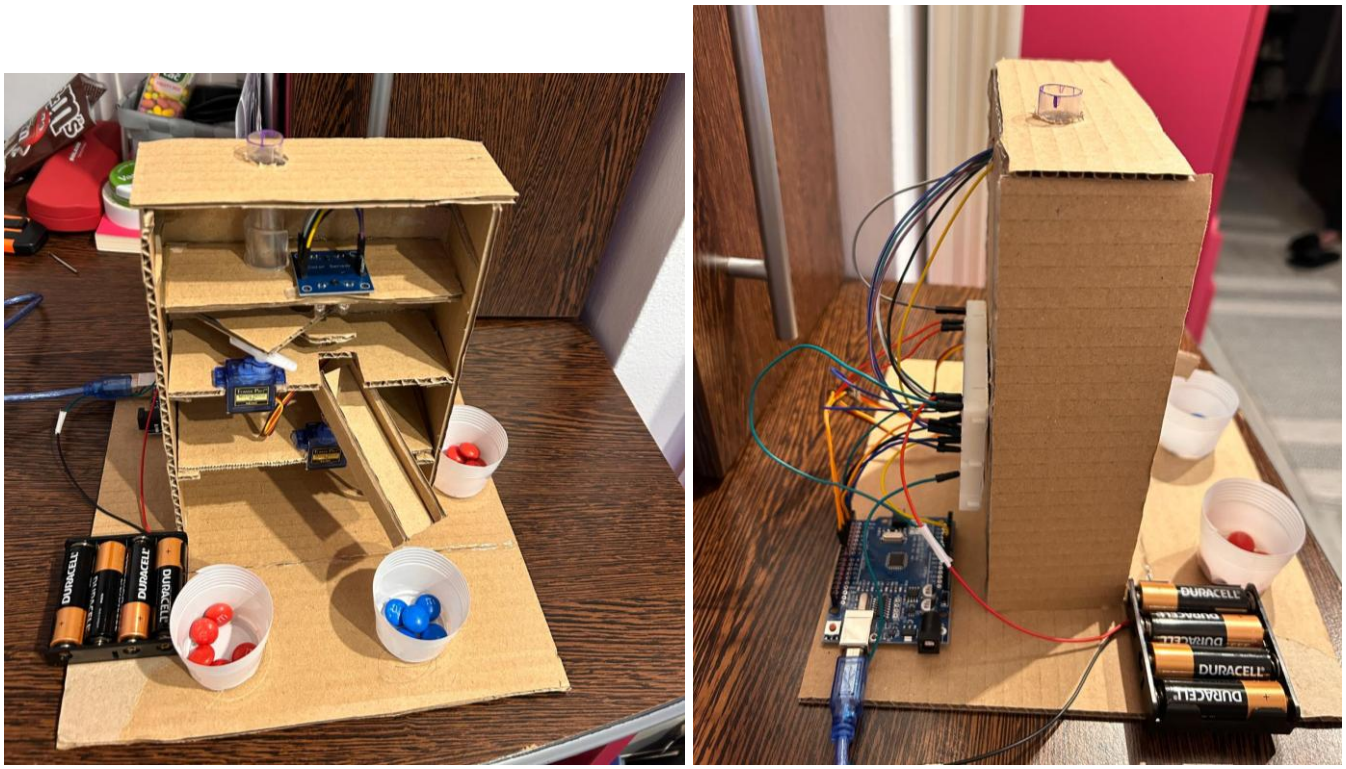


Figure 2- Real implementation of the candy sorting machine

The back of the box has the arduino along with a mini breadboard where all the connections were made and the power supply of the servo motors, which in this case were 4 batteries of 1.5V each.

WORKING PRINCIPLE:

-Initially, the M&M's are stored in the pipe and dropped into the platform attached to the top servo motor.

-The servo motor rotates and brings the candy to the color sensor that will detect its color.

-After the detection is made, the bottom servo will rotate to the particular position of the corresponding color and the top servo will rotate again until the candy drops down the guide rail.

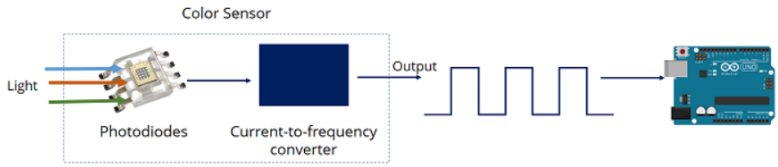
2. Theoretical Aspects

2.1 How does a TCS3200 work?

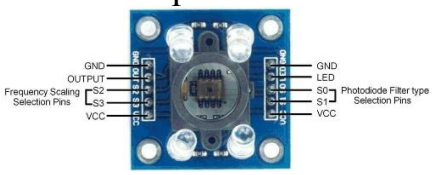
The TCS3200 has an array of photodiodes (semiconductors that convert light into current) with 4 different filters. The sensor has:

- 16 photodiodes with red filter – sensitive to red wavelength
- 16 photodiodes with red filter – sensitive to red wavelength
- 16 photodiodes with blue filter – sensitive to blue wavelength
- 16 photodiodes without filter

The sensor has a current-to-frequency converter that converts the photodiodes' readings into a square wave with a frequency that is proportional to the light intensity of the chosen color. This frequency is then, read by the Arduino.



Here is the pinout:



To select the color read by the photodiode, you use the control pins S2 and S3. As the photodiodes are connected in parallel, setting the S2 and S3 LOW and HIGH in different combinations allows you to select different photodiodes.

S0	S1	Output Frequency Scaling
L	L	Power down
L	H	2%
H	L	20%
H	H	100%

S2	S3	Photodiode Type
L	L	Red
L	H	Blue
H	L	Clear (no filter)
H	H	Green

Pins S0 and S1 are used for scaling the output frequency. Scaling the output frequency is useful to optimize the sensor readings for various frequency counters or microcontrollers. This will help us implement the Arduino code later for reading the colors.

2.1 How does a SG90 servo motor work?

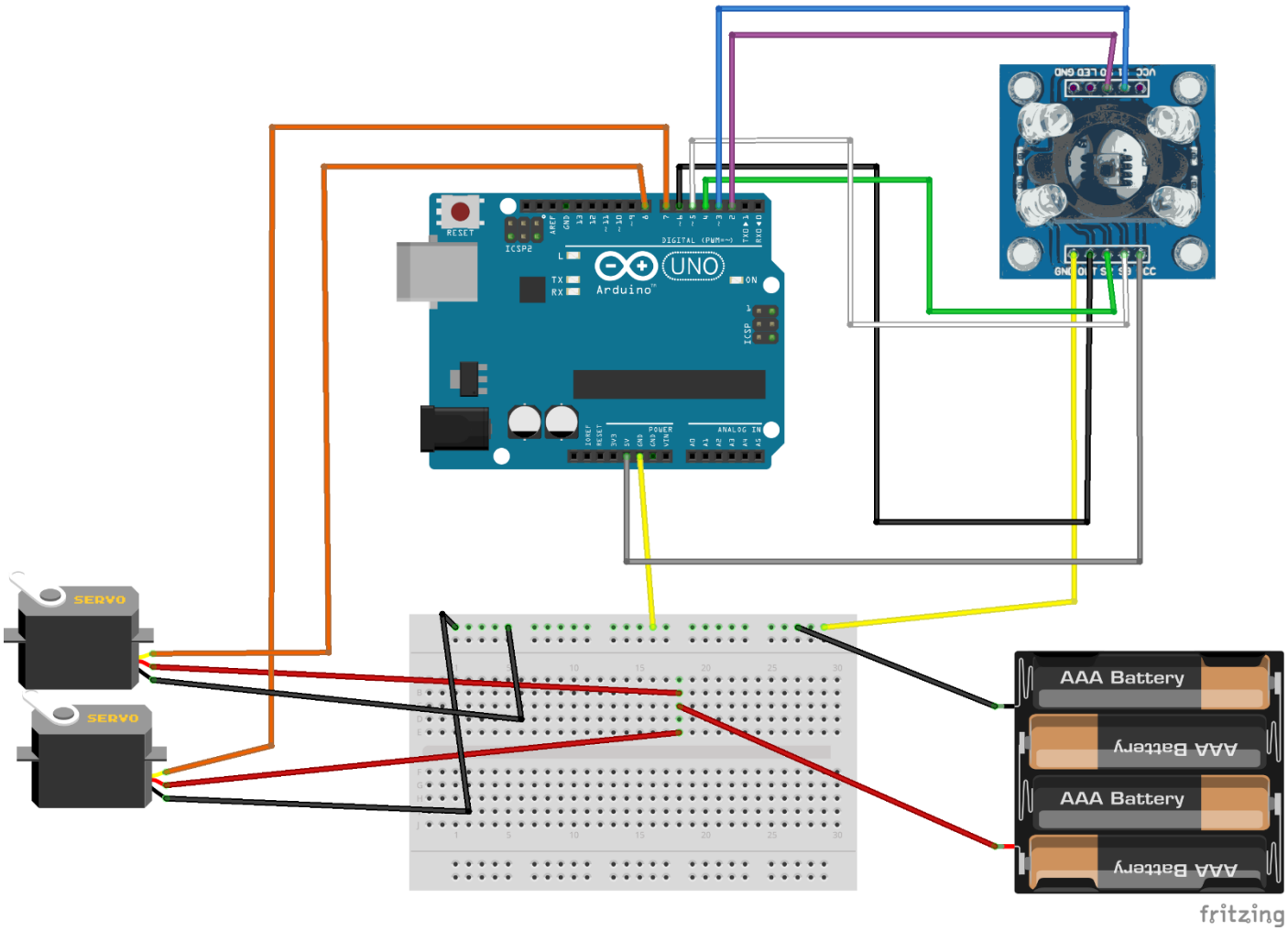
The SG90 servo motor operates using **Pulse Width Modulation (PWM)** within a closed-loop control system. As an analog micro servo, it adjusts its angular position based on variable **pulse widths** and continuously corrects itself to maintain the target angle. It can rotate up to 180 degrees.

CONTROL PINS:

- VCC (red): connected to the Arduino's 5V pin for power
- SIG (yellow): connected to the Arduino digital PWM pin
- GND (black): connected to the Arduino GND pin

3. Hardware implementation

The circuit schematic was made in the Fritzing App:



‼ Even if the theory says that we should connect the VCC from the servos to the 5V pin of the Arduino, when I did that none of the servos were functioning properly and they were making a weird noise.

The solution was to supply the servo motors from 4 batteries of 1,5V each, and the Arduino and the TCS3200 from the USB port via my computer.

4. Software implementation

```
#include <Servo.h>
#define S0 2
#define S1 3
#define S2 4
#define S3 5
#define sensorOut 6
Servo topServo;
Servo bottomServo;

int frequency = 0;
int color=0;
int unknownCount = 0;

void setup() {
  pinMode(S0, OUTPUT);      //initializing the pins of the arduino with the sensor
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(sensorOut, INPUT);

  // Setting  the frequency-scaling to 20%
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);
  topServo.attach(7);
  bottomServo.attach(8);
  Serial.begin(9600);
}
void loop() {
  topServo.write(115);
  delay(1000);

  for(int i = 115; i > 65; i--) {
    topServo.write(i);
    delay(2);
  }
  delay(1000);

  color = readColor();
  delay(100);
  switch (color) {
  case 1:
```

```
      Serial.println("Color detected: RED");
      bottomServo.write(105);
      unknownCount = 0;
      break;

    case 2:
      Serial.println("Color detected: BLUE");
      bottomServo.write(170);
      unknownCount = 0;
      break;

    default:
      Serial.println("Color detected: UNKNOWN");
      unknownCount++;
      break;
  }

  if (unknownCount >= 3) {
      Serial.println("STOP: 3 UNKNOWN DETECTED!!!!!!!!!! NO
CANDY LEFT!!!!!!!!");
    topServo.write(90);  // neutral position in the case of no candy recognised
    while (1);
  }

    delay(500);

    for(int i = 55; i > 45; i--) {
      topServo.write(i);
      delay(5);
    }
    delay(200);

    for(int i = 45; i < 80; i++) {
      topServo.write(i);
      delay(5);
    }
    color=0;
  }

  // Custom Function  for reading the colors using the sensor
  int readColor() {
    // Setting red filtered photodiodes to be read
```

```
  digitalWrite(S2, LOW);
  digitalWrite(S3, LOW);
  // Reading the output frequency
  frequency = pulseIn(sensorOut, LOW);
  int R = frequency;
  // Printing the value on the serial monitor
  Serial.print("R= ")://printing R
  Serial.print(frequency);//printing RED color frequency
  Serial.print("  ");
  delay(50);

  // Setting Green filtered photodiodes to be read
  digitalWrite(S2, HIGH);
  digitalWrite(S3, HIGH);
  // Reading the output frequency
  frequency = pulseIn(sensorOut, LOW);
  int G = frequency;
  // Printing the value on the serial monitor
  Serial.print("G= ")://printing G
  Serial.print(frequency);//printing RED color frequency
  Serial.print("  ");
  delay(70);

  // Setting Blue filtered photodiodes to be read
  digitalWrite(S2, LOW);
  digitalWrite(S3, HIGH);

  // Reading the output frequency
  frequency = pulseIn(sensorOut, LOW);
  int B = frequency;
  // Printing the value on the serial monitor
  Serial.print("B= ")://printing B
  Serial.print(frequency);//printing RED color frequency
  Serial.println("  ");
  delay(70);

 if (R >= 70 && R <= 100 && G >= 140 && G <= 165 && B >= 115 && B <= 150) {
   color = 1; // RED
 }
 else if (R >= 115 && R <= 150 && G >= 105 && G <= 135 && B >= 60  && B <= 95)
{
   color = 2; // BLUE
```

```
}
return color;
}
```

5. CONCLUSIONS

-the function for sorting the candy by color still needs to be calibrated every time we change the light in the room we want to use the Sorting Machine.

-After multiple tries I was capable only of sorting Blue M&M's from Red ones, because the RGB values for the other colors were too similar to each other and the surface of the candy is not flat and also shiny, so will not help us any better.

-Maybe if another algorithm of sorting was used then it would have been easier to detect multiple candy colors, but taking into consideration that the objects constantly move and even the slightest change in the Ambiental light can give erroneous values, I conclude that **TCS3200 is not the best sensor** for this kind of projects

What options do we have to make things work?

1. Another sensor, like **Adafruit TCS34725 RGB Color Sensor**

Key Features:

> Accurate color detection with built-in infrared filter
> 16-bit color depth for precise readings
> Simple to integrate with microcontrollers such as Arduino and Raspberry Pi
> Onboard I2C communication for quick data transmission

Pros:

> High sensitivity and color accuracy
> Performs effectively in various lighting conditions
> Suitable for advanced projects like color sorting or robotic vision systems

Cons:

> Slightly more expensive than other basic sensors
> Requires careful setup for calibration in certain lighting environments

2. Using LDRs and a RGB module + arduino

video_proiect.mp4

6. Bibliography

Written documentation:
https://docs.arduino.cc/tutorials/uno-rev3/intro-to-board/

https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial-tcs230-tcs3200-color-sensor/

https://eddragon.com/sg90-servo-motor-basics/

Youtube:

https://www.youtube.com/watch?v=2p3EqUVDb0A

https://www.youtube.com/watch?v=ZOllXMxLRqc